

LAB ASSIGNMENT -2

1. Create a structure in C++ containing the details of Students as details below and a main function to execute the structure.

Data Members(properties):

Name

Roll No

Degree

Hostel

CurrentCGPA

Member Function(behavior):

addDetails();

updateDetails();

updateCGPA();

updateHostel();

displaydetails();

Code:

```
#include <iostream>
using namespace std;
struct Student
{
    string name;
    int rollNo;
    string degree;
    string hostel;
    float currentCGPA;
    void addDetails()
    {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Roll No: ";
        cin >> rollNo;
        cout << "Enter Degree: ";
        cin >> degree;
```

```

        cout << "Enter Hostel: ";
        cin >> hostel;
        cout << "Enter CGPA: ";
        cin >> currentCGPA;
    }
    void updateDetails()
    {
        cout << "Enter New Name: ";
        cin >> name;
        cout << "Enter New Degree: ";
        cin >> degree;
    }
    void updateCGPA()
    {
        cout << "Enter New CGPA: ";
        cin >> currentCGPA;
    }
    void updateHostel()
    {
        cout << "Enter New Hostel: ";
        cin >> hostel;
    }
    void displayDetails()
    {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "Degree: " << degree << endl;
        cout << "Hostel: " << hostel << endl;
        cout << "CGPA: " << currentCGPA << endl;
    }
};

int main()

```

```

{
    Student s;
    s.addDetails();
    s.displayDetails();
    s.updateCGPA();
    s.updateHostel();
    s.displayDetails();
    return 0;
}

```

2. Differentiate between private and public access/scope. Perform the question no. 1 with class instead of structure with having the data members private and some member functions in private scope and some in public scope.

Code:

```
#include <iostream>
```

```
using namespace std;
```

```
class Student
```

```
{
```

```
private:
```

```
    string name;
```

```
    int rollNo;
```

```
    string degree;
```

```
    string hostel;
```

```
    float cgpa;
```

```
    void secretFunction()
```

```
{
```

```
    cout << "Private Function Called\n";
```

```
}
```

public:

void addDetails()

{

cout << "Enter Name: ";

cin >> name;

cout << "Enter Roll No: ";

cin >> rollNo;

cout << "Enter Degree: ";

cin >> degree;

cout << "Enter Hostel: ";

cin >> hostel;

cout << "Enter CGPA: ";

cin >> cgpa;

}

void updateCGPA()

{

cout << "Enter New CGPA: ";

cin >> cgpa;

}

void displayDetails()

{

secretFunction();

cout << "\nStudent Details\n";

```

        cout << "Name: " << name << endl;

        cout << "Roll No: " << rollNo << endl;

        cout << "Degree: " << degree << endl;

        cout << "Hostel: " << hostel << endl;

        cout << "CGPA: " << cgpa << endl;

    }

};

int main()

{

    Student s;

    s.addDetails();

    s.updateCGPA();

    s.displayDetails();

    return 0;

}

```

3. Create a code snippet that illustrates the following: Calling of private member functions inside public member function

```

Code:
#include <iostream>

using namespace std;

class Demo

{

private:

    void showMessage()

```

```

    {

        cout << "Private Function Called\n";

    }

public:

    void display()

    {

        showMessage();

    }

};

int main()

{

    Demo d;

    d.display();

    return 0;

}

```

4. Define a class Rectangle with variables width and height of integer typwe along with following:
 - a. void getdata() to initialize object values
 - b. void calculatearea() to calculate and display the area.

Code:

```
#include <iostream>
```

```
using namespace std;
```

```
class Rectangle
```

```
{
```

private:

int width, height;

public:

void getdata()

{

cout << "Enter Width: ";

cin >> width;

cout << "Enter Height: ";

cin >> height;

}

void calculatearea()

{

int area = width * height;

cout << "Area = " << area << endl;

}

};

int main()

{

Rectangle r;

r.getdata();

r.calculatearea();

return 0;

}

5. Define a class Complex with variables real and imaginary along with following:

- (a) void setComplex (float, float) to initialize object values.
- (b) void displayComplex() to show the complex number
- (c) Pass and return objects to calculate sum of two complex numbers. Display the sum.

Code:

```
#include <iostream>

using namespace std;

class Complex
{
private:
    float real, imaginary;
public:
    void setComplex(float r, float i)
    {
        real = r;
        imaginary = i;
    }
    void displayComplex()
    {
        cout << real << " + " << imaginary << "i" << endl;
    }
    Complex addComplex(Complex c)
    {
        Complex temp;

        temp.real = real + c.real;
        temp.imaginary = imaginary + c.imaginary;
        return temp;
    }
};

int main()
{
```

```

Complex c1, c2, sum;
c1.setComplex(2, 3);
c2.setComplex(4, 5);
sum = c1.addComplex(c2);
cout << "First Complex Number: ";
c1.displayComplex();
cout << "Second Complex Number: ";
c2.displayComplex();
cout << "Sum: ";
sum.displayComplex();
return 0;
}

```

6. Implement *scope resolution operator* : : for the following uses:

- (a) Class functions defined outside the class
- (b) Access a global variable with same name as a local variable
- (c) Access a static variables
- (d) Use inbuilt libraries (cin cout with scope resolution operator)

Code:

a)

```

#include <iostream>

using namespace std;

class Demo
{
public:
    void show();
};

void Demo::show()
{
    cout << "Function Defined Outside Class";
}

int main()
{
    Demo d;
}

```

```
    d.show();  
    return 0;  
}
```

b)

```
#include <iostream>  
using namespace std;  
int x = 50;  
int main()  
{  
    int x = 10;  
    cout << "Local x = " << x << endl;  
    cout << "Global x = " << ::x << endl;  
    return 0;  
}
```

c)

```
#include <iostream>  
using namespace std;  
class Test  
{  
public:  
    static int count;  
};  
int Test::count = 100;  
int main()  
{  
    cout << Test::count;  
    return 0;  
}
```

d)

```
#include <iostream>  
int main()  
{
```

```
    std::cout << "Hello World";  
    return 0;  
}
```

7. Create a code to implement the *namespace* and use similar variables and functions defined in different code sections.

Code:

```
#include <iostream>  
  
using namespace std;  
  
namespace First  
{  
    int value = 10;  
    void display()  
    {  
        cout << "First Namespace\n";  
    }  
}  
  
namespace Second  
{  
    int value = 20;  
    void display()  
    {  
        cout << "Second Namespace\n";  
    }  
}  
  
int main()  
{  
    cout << First::value << endl;  
    First::display();  
    cout << Second::value << endl;  
    Second::display();  
    return 0;  
}
```